

Client-Centric Consistency in a Replicated MongoDB Database

Experiments under replica lag, node failure, and network partition

Course	DSA5208 Distributed Systems
Database	MongoDB Community 7.0.32
Deployment	Three Docker containers, one replica set
Experiment date	17 September 2026
Group	<i>Add group members and student IDs before submission</i>

Submission note. The report is generated from an audited formal run containing 208 client histories and 856 database commands. The accompanying archive includes the raw histories, three server logs, metadata, checksums, and reproduction scripts.

Abstract

This project evaluates four client-centric consistency properties—read-your-writes, monotonic reads, monotonic writes, and writes-follow-reads—in a three-member MongoDB replica set. We vary causal history tracking, read concern, and write concern across four client configurations. We then execute the same application history during normal operation, deliberate replica lag, secondary and primary crashes, a typical 2+1 network partition, complete 1+1+1 isolation, and a partition followed by rollback. The formal run contains 208 histories and 856 instrumented commands. Weak or non-causal configurations returned stale values under lag and partition, while causally tracked configurations waited or failed rather than returning values older than their causal lower bound. A 2+1 partition additionally showed that the majority side remained writable while the isolated minority rejected writes after its former primary stepped down. The observations agree with the predicted distinction between consistency of successful observations and availability.

Contents

1	System and deployment	2
1.1	Chosen database	2
1.2	Architecture	2
1.3	Installation and startup	2
2	Consistency configurations and predictions	2
3	Experimental method	3
3.1	Workload and oracle	3
3.2	Scenarios	4
4	Results	4
4.1	The 2+1 partition	4
4.2	Complete isolation and rollback	5
5	Comparison with predictions	5
6	Limitations	6
7	Reproducibility and submitted evidence	6
8	AI usage disclosure	6
A	Exact configuration summary	7

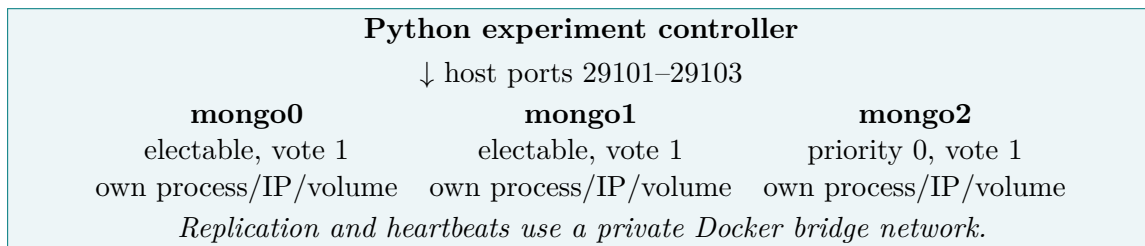
1 System and deployment

1.1 Chosen database

MongoDB is a document database whose replica sets provide automatic primary election, asynchronous replication, configurable read and write concerns, and causally consistent client sessions. These mechanisms make it possible to vary consistency and durability on a per-operation basis while preserving the same application workload [1, 2, 3].

The experiment uses MongoDB Community 7.0.32 and PyMongo 4.10.1. All versions are pinned in the Dockerfile and `requirements.txt`. The storage engine is WiredTiger.

1.2 Architecture



Each container stores a complete data copy in a named volume. All three members vote, but `mongo2` has priority zero so that it cannot become primary. Nodes 0 and 1 are electable. The replica set therefore survives one node failure and can elect a primary on the two-node side of a 2+1 split. The controller uses direct connections so that each read target is known. Causal metadata is explicitly transferred when the logical client changes physical connections; a PyMongo session is never passed to a different `MongoClient`.

This is a distributed logical deployment using multiple containers on one physical Mac, as permitted by the assignment. It does not emulate independent power, disks, or host kernels.

1.3 Installation and startup

Docker Desktop and Docker Compose are required. From the repository root:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m unittest -v test_checker.py
.venv/bin/python run.py --work-dir work/full --out results/full
.venv/bin/python analyze.py results/full
```

The runner builds the pinned MongoDB image, creates fresh containers and volumes, calls `replSetInitiate`, waits for one primary and two secondaries, performs the tests, heals all partitions, collects logs, and stops the containers. New directories are required for every run, preventing accidental reuse of earlier data.

2 Consistency configurations and predictions

MongoDB read concern specifies which data a read may expose, while write concern specifies how many voting members must acknowledge a write. A causally consistent session carries logical time from an earlier operation to a later operation. For reads, the driver sends an `afterClusterTime` lower bound so that a server cannot return an older snapshot merely to respond quickly [1, 7].

Table 1: Client configurations.

ID	Causal tracking	Read concern	Write concern
A	off	local	$w : 1$
B	off	majority	$w : \text{majority}$
C	on	majority	$w : 1$
D	on	majority	$w : \text{majority}$

Table 2 records the predictions made from the documented causal-session matrix. G means a documented guarantee for successful operations under the stated settings; NG means that a permitted execution can violate the property. An NG prediction does not mean every execution must show a violation.

Table 2: Predicted client-centric guarantees.

Configuration	RYW	MR	MW	WFR
A: local/ $w : 1$, causal off	NG	NG	NG	NG
B: majority/majority, causal off	NG	NG	NG	NG
C: majority/ $w : 1$, causal on	NG	G	NG	G
D: majority/majority, causal on	G	G	G	G

The report uses a durable-history interpretation: if a write returned an acknowledgment, later loss of that write during rollback can witness an RYW or MW violation. MongoDB also documents a weaker interpretation in which rollback is itself part of the history. We state the chosen interpretation to avoid mixing the two definitions [1, 6].

3 Experimental method

3.1 Workload and oracle

Every trial uses a unique document initialized on all three members as $\{a:0, b:0\}$. The usual operation order is:

W1: set $a=1 \rightarrow R1 \rightarrow R2 \rightarrow W2$: set $b=1$ and return pre-image.

`findOneAndUpdate(..., returnDocument=BEFORE)` makes W2 a real modifying write while atomically revealing the database state on which it executed. This avoids incorrectly inferring write ordering from a later final value.

Table 3: Operational checks for the four properties.

Property	Eligible check	Violation witness
Read-your-writes	W1 acknowledged and at least one later read returned	Any successful read invoked after W1 completed returns $a < 1$
Monotonic reads	R1 and R2 both returned	$R2.a < R1.a$
Monotonic writes	W1 and W2 acknowledged	W2 pre-image has $a < 1$
Writes-follow-reads	W2 acknowledged after a successful read	W2 pre-image is older than the maximum version observed by earlier reads

Read errors expose no value and therefore produce an inconclusive read-to-read check. A write timeout can have an unknown outcome and is not treated as an acknowledged predecessor. Errors and timeouts are reported separately from stale-value violations. Eight synthetic unit tests validate preserved histories, stale reads, timeout handling, rollback, WFR order, and the requirement that RYW examine every successful read following W1.

3.2 Scenarios

Table 4: Operating scenarios and repetitions per configuration.

Scenario	Fault and routing	n
Normal	No injected fault; R2 targets a known secondary.	20
Replica lag	mongo2 applies operations with a three-second delay; R2 targets it.	10
Secondary crash	Kill mongo2 with SIGKILL; the surviving pair remains available.	10
Primary crash	W1/R1, kill primary, await election, then R2/W2 on the new primary.	3
2+1 partition	Isolate the old primary; the other two nodes elect a new primary. W1/R1/W2 use the majority side and R2 probes the minority.	3
1+1+1 isolation	Drop all peer traffic while host-to-container client access remains.	3
Partition + rollback	Isolate before W1, kill old primary, heal survivors, elect a new primary, then execute W2/R2.	3

Faults are implemented with container-local `iptables` INPUT/OUTPUT rules using `NET_ADMIN`. Rules block only peer container addresses; the controller retains access through published loopback ports. Every disruptive trial receives its own cut/heal cycle and an administrative $w : 3$ recovery barrier. The audit validates actual wire commands, read/write concerns, causal bounds, unique trial IDs, and the expected topology of every partition snapshot.

4 Results

The formal run completed 208 histories and 856 application commands. All 856 commands had a corresponding wire-level instrumentation event. Twenty-four isolated topology snapshots were captured. The audit found no incorrect concern parameters and no network-topology failures. In the following tables, cells are *violations/eligible checks*; “—” means no eligible check. The final column is operation errors/attempts.

Under replica lag, A and B returned the stale value in every eligible RYW and MR check. Majority visibility alone did not preserve this client’s position in history. In contrast, C and D sent a causal lower bound; R2 waited until its 700 ms deadline and returned an error rather than an older value. R1 remained a successful post-W1 read, so RYW was eligible and non-violating, while MR was inconclusive because R2 returned no value.

4.1 The 2+1 partition

The typical partition produced the clearest availability contrast. After the old primary was isolated, it stepped down and the two-node side elected a new primary. That majority side successfully

Table 5: Normal operation, lag, and node-failure results.

Scenario	Config	RYW	MR	MW	WFR	Errors
Normal	A	0/20	0/20	0/20	0/20	0/80
Normal	B	0/20	0/20	0/20	0/20	0/80
Normal	C	0/20	0/20	0/20	0/20	0/80
Normal	D	0/20	0/20	0/20	0/20	0/80
Replica lag	A	10/10	10/10	0/10	0/10	0/40
Replica lag	B	10/10	10/10	0/10	0/10	0/40
Replica lag	C	0/10	–	0/10	0/10	10/40
Replica lag	D	0/10	–	0/10	0/10	10/40
Secondary crash	A–D	0/10 each	0/10 each	0/10 each	0/10 each	0/40 each
Primary crash	A–D	0/3 each	0/3 each	0/3 each	0/3 each	0/12 each

Table 6: Network-partition and rollback results.

Scenario	Config	RYW	MR	MW	WFR	Errors
2+1 partition	A	3/3	3/3	0/3	0/3	3/15
2+1 partition	B	3/3	3/3	0/3	0/3	3/15
2+1 partition	C	0/3	–	0/3	0/3	6/15
2+1 partition	D	0/3	–	0/3	0/3	6/15
1+1+1 isolation	A	3/3	3/3	0/3	0/3	3/15
1+1+1 isolation	B	–	0/3	–	–	9/15
1+1+1 isolation	C	–	–	0/3	–	9/15
1+1+1 isolation	D	–	–	–	–	15/15
Partition + rollback	A	3/3	3/3	3/3	3/3	0/12
Partition + rollback	B	–	0/3	–	0/3	3/12
Partition + rollback	C	3/3	–	3/3	–	3/12
Partition + rollback	D	–	–	–	–	6/12

acknowledged W1 and W2, including majority writes. The minority side remained directly readable by the controller. A and B accepted the stale minority read, producing RYW and MR witnesses in all three repetitions. C and D refused to return an older snapshot: their minority reads reached the deadline, but their majority-side writes still succeeded. The final minority-side write probe failed for every configuration after step-down.

4.2 Complete isolation and rollback

With no pair of voters connected, a majority could not form. Configuration D consequently had no successful measured operations in the complete-isolation trials. This is an availability outcome, not a consistency violation. Weak configuration A could briefly observe stale data while the old primary detected quorum loss.

In the rollback scenario, an isolated $w : 1$ W1 could be acknowledged and later disappear from the elected branch. Configuration A then witnessed all four violations. Configuration C retained causal read constraints but could still lose a non-durable $w : 1$ predecessor, producing RYW and MW witnesses under the report’s durable-history interpretation.

5 Comparison with predictions

The observations agree with the predictions:

- D produced no witnessed violation. When its required history or quorum was unavailable, the operation waited or failed rather than returning a contradictory value.

- C preserved MR and WFR for successful operations, but a rolled-back $w : 1$ predecessor can violate durable-history RYW and MW.
- A exposed stale reads during lag and both partition types, and exposed all four anomalies after rollback.
- B demonstrated that majority read/write concerns without causal history tracking do not by themselves guarantee RYW or MR when a client reads from a lagging or isolated member.

Zero observed violations are not proofs of universal guarantees. Guarantees come from the documented model; executions supply corroborating examples or counterexamples. Likewise, timeouts are evidence about availability rather than successful stale observations.

6 Limitations

1. All containers share one Mac and Docker's Linux VM, so host, kernel, disk, and power failures are correlated.
2. Direct connections intentionally force particular read targets. The experiment does not characterize normal driver load balancing.
3. A three-second delayed member is a laboratory control, not a realistic WAN model.
4. Each trial uses one document, a binary monotone version field, and sequential client operations. Cross-document causality, sharding, transactions, and concurrent overwrites are outside scope.
5. Fault scenarios use three deterministic repetitions per configuration. They are suitable for reproducible witnesses, not production-frequency estimates.
6. The four property checks overlap within a history and must not be summed as statistically independent samples.
7. The durable-history treatment of rollback is explicit but not the only terminology used in MongoDB documentation.

7 Reproducibility and submitted evidence

The repository contains `compose.yaml`, `Dockerfile`, `lab.py`, `docker_backend.py`, `analyze.py`, checker unit tests, dependency pins, and a README. The archive `docker-run-2026-09-17-raw.zip` contains:

- `history.jsonl`: operations, wire commands, returned values, causal tokens, fault events, topology snapshots, and recovery observations;
- `trials.json`: structured histories and four-property classifications;
- three `mongod-*.log` files containing elections, replication, and rollback;
- `metadata.json`, `audit.json`, and derived summaries.

`RAW_DATA_MANIFEST.json` supplies SHA-256 digests for every original file and the ZIP. Re-analysis must report 208 histories, 856 operations and wire commands, 24 isolation snapshots, and zero parameter or isolation audit failures.

8 AI usage disclosure

OpenAI Codex assisted with interpreting the assignment and MongoDB documentation, designing the experiment matrix, implementing and debugging the harness, executing local Docker tests, auditing raw evidence, and drafting this report. Database measurements were produced by real MongoDB processes. The group remains responsible for reviewing the code, validating the results, adding member information, and submitting the final work.

References

- [1] MongoDB, “Causal Consistency and Read and Write Concerns,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/core/causal-consistency-read-write-concerns/>
- [2] MongoDB, “Read Isolation, Consistency, and Recency,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/core/read-isolation-consistency-recency/>
- [3] MongoDB, “Write Concern,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/reference/write-concern/>
- [4] MongoDB, “Deploy a Self-Managed Replica Set for Testing and Development,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/tutorial/deploy-replica-set-for-testing/>
- [5] MongoDB, “Delayed Replica Set Members,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/core/replica-set-delayed-member/>
- [6] MongoDB, “Rollbacks During Replica Set Failover,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/core/replica-set-rollbacks/>
- [7] MongoDB, “PyMongo ClientSession API,” PyMongo 4.10.1. https://pymongo.readthedocs.io/en/4.10.1/api/pymongo/client_session.html
- [8] Docker, “Compose file reference: services.” <https://docs.docker.com/reference/compose-file/services/>
- [9] DSA5208, “Lecture 3: Consistency Models,” course lecture slides, 2026.

A Exact configuration summary

MongoDB runs with `-replSet cc_lab`, `-bind_ip_all`, a 128 MB oplog, and a 0.25 GB WiredTiger cache per node. The election timeout is 15 seconds and heartbeat interval is 500 ms. Measured reads use a 700 ms server deadline, write concern timeout is 900 ms, driver socket timeout is 2500 ms, and retryable reads/writes are disabled. The random configuration order uses seed 419. Administrative recovery barriers deliberately use longer timeouts than measured application operations.